

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE

SUBJECT : Cyber & Information Security

Aim: Implement digital signature algorithms such as RSA-based signatures, and verify the integrity and authenticity of digitally signed messages.

INPUT

```
from cryptography.hazmat.primitives.asymmetric import rsa, padding
from cryptography.hazmat.primitives import hashes

# Generate RSA keys
private_key = rsa.generate_private_key(
    public_exponent=65537,
    key_size=2048
)

public_key = private_key.public_key()

# Take message from user
message = input("Enter the message: ").encode()

# Create digital signature
signature = private_key.sign(
    message,
    padding.PSS(
        mgf=padding.MGF1(hashes.SHA256()),
        salt_length=padding.PSS.MAX_LENGTH
    ),
    hashes.SHA256()
)

print("\nDigital Signature Created Successfully!")

# Verify original message
try:
    public_key.verify(
        signature,
        message,
        padding.PSS(
            mgf=padding.MGF1(hashes.SHA256()),
            salt_length=padding.PSS.MAX_LENGTH
```

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE

SUBJECT : Cyber & Information Security

```
    ),
    hashes.SHA256()
)

print("Signature Verification: SUCCESS")
print("Message is authentic and unchanged.")

except:
    print("Signature Verification: FAILED")

# Ask user for message to verify
verify_message = input("\nEnter message to verify: ").encode()

# Verify entered message
try:
    public_key.verify(
        signature,
        verify_message,
        padding.PSS(
            mgf=padding.MGF1(hashes.SHA256()),
            salt_length=padding.PSS.MAX_LENGTH
        ),
        hashes.SHA256()
    )

    print("Verification SUCCESS")
    print("Message is authentic and unchanged.")

except:
    print("Verification FAILED")
    print("Message has been modified.")
```

rsa.generate_private_key() – Generates the RSA private key.

private_key.public_key() – Generates the public key from the private key.

input() – Takes the message from the user.

private_key.sign() – Creates a digital signature for the message.

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE

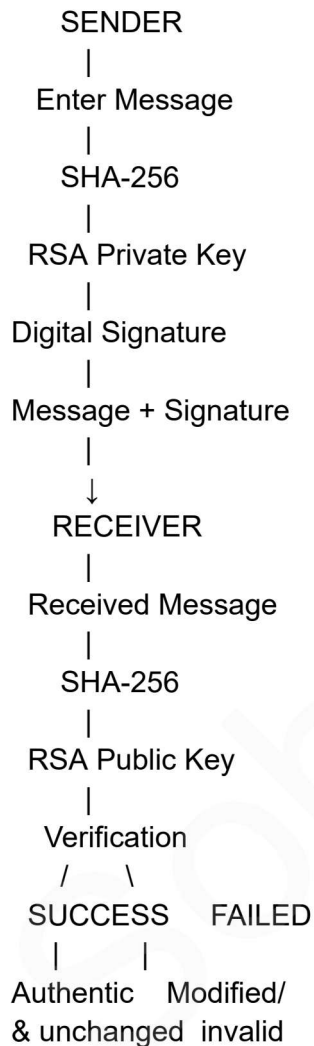
SUBJECT : Cyber & Information Security

`hashes.SHA256()` – Creates a secure hash of the message.

`public_key.verify()` – Verifies whether the message matches the digital signature.

`try-except` – Handles successful and failed signature verification.

complete process is:

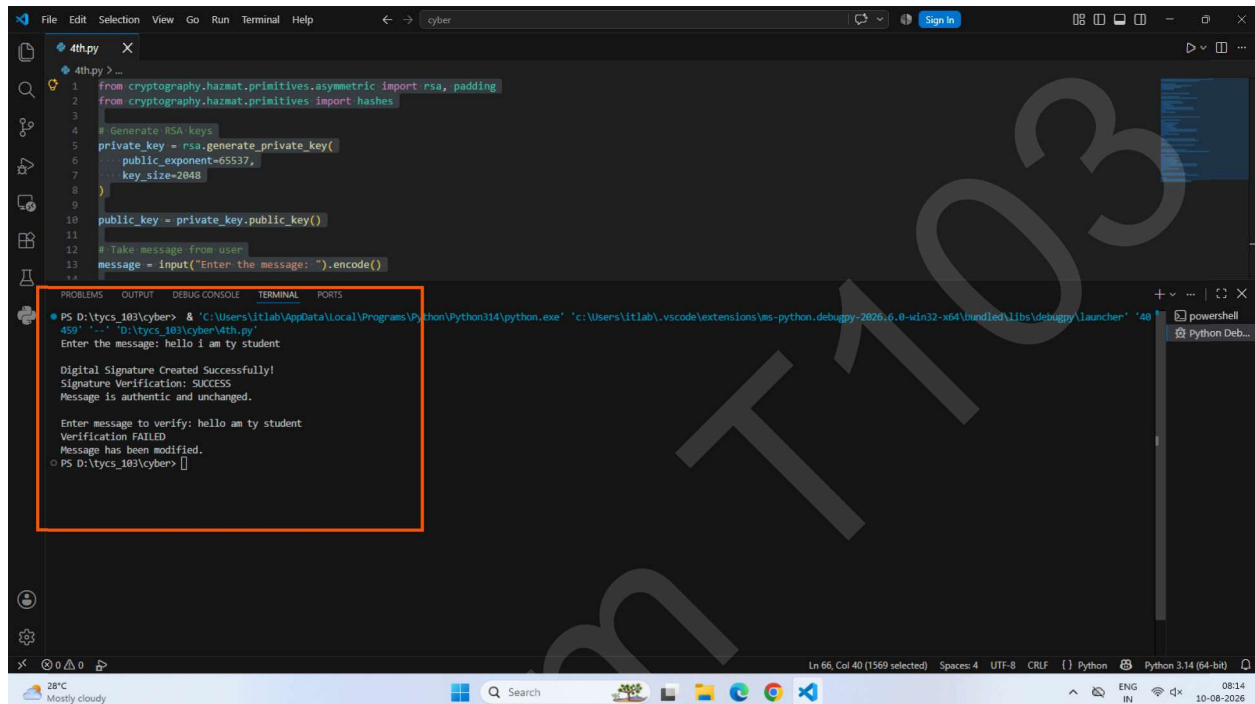


RSA digital signature uses the private key to sign a message and the corresponding public key to verify it. SHA-256 is used to create a hash, so if the message is modified, its hash changes and the signature verification fails.

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE

SUBJECT : Cyber & Information Security

OUTPUT

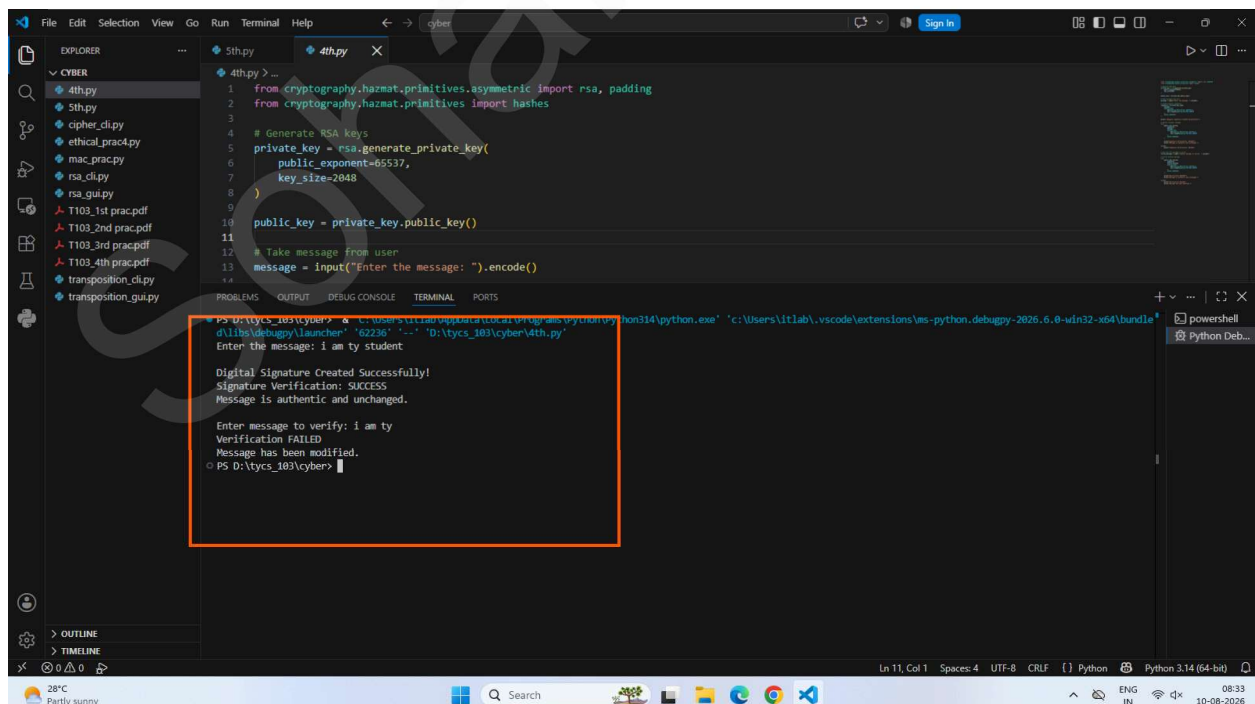


```
File Edit Selection View Go Run Terminal Help
4th.py
1 from cryptography.hazmat.primitives.asymmetric import rsa, padding
2 from cryptography.hazmat.primitives import hashes
3
4 # Generate RSA keys
5 private_key = rsa.generate_private_key(
6     public_exponent=65537,
7     key_size=2048
8 )
9
10 public_key = private_key.public_key()
11
12 # Take message from user
13 message = input("Enter the message: ").encode()
14
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS D:\tycs_103\cyber> & 'C:\Users\lilab\AppData\Local\Programs\Python\Python314\python.exe' 'c:\Users\lilab\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundle\launcher' '40'
459' '-' 'D:\tycs_103\cyber\4th.py'
Enter the message: hello i am ty student

Digital Signature Created Successfully!
Signature Verification: SUCCESS
Message is authentic and unchanged.

Enter message to verify: hello am ty student
Verification FAILED!
Message has been modified.
PS D:\tycs_103\cyber>
```

Verification Failed



```
File Edit Selection View Go Run Terminal Help
EXPLORER
CYBER
4th.py
5th.py
cipher_cli.py
ethical_prac4.py
mac_prac.py
rsa_cli.py
rsa_gui.py
T103_1st prac.pdf
T103_2nd prac.pdf
T103_3rd prac.pdf
T103_4th prac.pdf
transposition_cli.py
transposition_gui.py
4th.py
1 from cryptography.hazmat.primitives.asymmetric import rsa, padding
2 from cryptography.hazmat.primitives import hashes
3
4 # Generate RSA keys
5 private_key = rsa.generate_private_key(
6     public_exponent=65537,
7     key_size=2048
8 )
9
10 public_key = private_key.public_key()
11
12 # Take message from user
13 message = input("Enter the message: ").encode()
14
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS D:\tycs_103\cyber> & 'C:\Users\lilab\AppData\Local\Programs\Python\Python314\python.exe' 'c:\Users\lilab\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundle\launcher' '62236' '-' 'D:\tycs_103\cyber\4th.py'
Enter the message: i am ty student

Digital Signature Created Successfully!
Signature Verification: SUCCESS
Message is authentic and unchanged.

Enter message to verify: i am ty
Verification FAILED
Message has been modified.
PS D:\tycs_103\cyber>
```

Soham Patil
Tycs T103

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE

SUBJECT : Cyber & Information Security

Input Gui

```
import tkinter as tk
from tkinter import messagebox
from cryptography.hazmat.primitives.asymmetric import rsa, padding
from cryptography.hazmat.primitives import hashes

# ----- RSA Functions -----

private_key = None
public_key = None
signature = None

def generate_keys():
    global private_key, public_key

    private_key = rsa.generate_private_key(
        public_exponent=65537,
        key_size=2048
    )

    public_key = private_key.public_key()

    status_label.config(
        text="RSA keys generated successfully.",
        fg="green"
    )

def sign_message():
    global signature

    if private_key is None:
        messagebox.showerror(
            "Error",
            "Please generate RSA keys first."
```

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE

SUBJECT : Cyber & Information Security

```
)  
    return  
  
message = message_entry.get()  
  
if message == "":  
    messagebox.showerror(  
        "Error",  
        "Please enter a message."  
    )  
    return  
  
# Create digital signature  
signature = private_key.sign(  
    message.encode(),  
    padding.PSS(  
        mgf=padding.MGF1(hashes.SHA256()),  
        salt_length=padding.PSS.MAX_LENGTH  
    ),  
    hashes.SHA256()  
)  
  
signature_text.delete("1.0", tk.END)  
signature_text.insert(  
    tk.END,  
    signature.hex()[:100] + "..."  
)  
  
status_label.config(  
    text="Digital signature created successfully",  
    fg="green"  
)  
  
def verify_signature():  
    if public_key is None:  
        messagebox.showerror(  

```

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE

SUBJECT : Cyber & Information Security

```
"Error",
    "Please generate RSA keys first."
)
return

if signature is None:
    messagebox.showerror(
        "Error",
        "Please sign the message first."
    )
    return

message = verify_entry.get()

if message == "":
    messagebox.showerror(
        "Error",
        "Please enter a message to verify."
    )
    return

try:
    public_key.verify(
        signature,
        message.encode(),
        padding.PSS(
            mgf=padding.MGF1(hashes.SHA256()),
            salt_length=padding.PSS.MAX_LENGTH
        ),
        hashes.SHA256()
    )

    result_label.config(
        text="✓ SIGNATURE VERIFIED\n"
        "Message is authentic and unchanged.",
        fg="green"
    )
```

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE

SUBJECT : Cyber & Information Security

```
except Exception:
    result_label.config(
        text="X SIGNATURE VERIFICATION FAILED\n"
        "Message has been modified or signature is invalid.",
        fg="red"
    )

def clear_all():
    global private_key, public_key, signature

    private_key = None
    public_key = None
    signature = None

    message_entry.delete(0, tk.END)
    verify_entry.delete(0, tk.END)
    signature_text.delete("1.0", tk.END)

    status_label.config(
        text="",
        fg="black"
    )

    result_label.config(
        text="",
        fg="black"
    )

# ----- Main Window -----

root = tk.Tk()
root.title("RSA Digital Signature")
root.geometry("750x650")
root.resizable(False, False)
```

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE

SUBJECT : Cyber & Information Security

```
root.configure(bg="#f2f4f7")

# ----- Title -----

title = tk.Label(
    root,
    text="RSA Digital Signature",
    font=("Arial", 22, "bold"),
    bg="#f2f4f7",
    fg="#1f2937"
)
title.pack(pady=15)

subtitle = tk.Label(
    root,
    text="Message Integrity and Authenticity Verification",
    font=("Arial", 11),
    bg="#f2f4f7",
    fg="#6b7280"
)
subtitle.pack()

# ----- Key Section -----

key_frame = tk.LabelFrame(
    root,
    text=" RSA Key Generation ",
    font=("Arial", 12, "bold"),
    bg="white",
    padx=20,
    pady=15
)
key_frame.pack(
    padx=30,
    pady=15,
```

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE

SUBJECT : Cyber & Information Security

```
        fill="x"
    )

    tk.Label(
        key_frame,
        text="RSA Key Size: 2048 bits",
        font=("Arial", 11),
        bg="white"
    ).pack(side="left", padx=10)

    tk.Button(
        key_frame,
        text="Generate RSA Keys",
        command=generate_keys,
        font=("Arial", 10, "bold"),
        padx=15,
        pady=7
    ).pack(side="right", padx=10)

# ----- Message Section -----

message_frame = tk.LabelFrame(
    root,
    text=" Message ",
    font=("Arial", 12, "bold"),
    bg="white",
    padx=20,
    pady=15
)
message_frame.pack(
    padx=30,
    pady=5,
    fill="x"
)

tk.Label(
```

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE

SUBJECT : Cyber & Information Security

```
message_frame,
text="Enter message:",
font=("Arial", 10),
bg="white"
).pack(anchor="w")

message_entry = tk.Entry(
    message_frame,
    font=("Arial", 11)
)
message_entry.pack(
    fill="x",
    pady=8
)

tk.Button(
    message_frame,
    text="Sign Message",
    command=sign_message,
    font=("Arial", 10, "bold"),
    padx=20,
    pady=7
).pack()

# ----- Signature Section -----

signature_frame = tk.LabelFrame(
    root,
    text=" Digital Signature ",
    font=("Arial", 12, "bold"),
    bg="white",
    padx=20,
    pady=10
)
signature_frame.pack(
    padx=30,
```

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE

SUBJECT : Cyber & Information Security

```
pady=10,  
fill="x"  
)  
  
signature_text = tk.Text(  
    signature_frame,  
    height=3,  
    font=("Consolas", 9),  
    wrap="word"  
)  
signature_text.pack(fill="x")  
  
# ----- Verification Section -----  
  
verify_frame = tk.LabelFrame(  
    root,  
    text=" Verify Message ",  
    font=("Arial", 12, "bold"),  
    bg="white",  
    padx=20,  
    pady=15  
)  
verify_frame.pack(  
    padx=30,  
    pady=5,  
    fill="x"  
)  
  
tk.Label(  
    verify_frame,  
    text="Enter message to verify:",  
    font=("Arial", 10),  
    bg="white"  
)  
)  
.pack(anchor="w")  
  
verify_entry = tk.Entry(  

```

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE

SUBJECT : Cyber & Information Security

```
verify_frame,
font=("Arial", 11)
)
verify_entry.pack(
    fill="x",
    pady=8
)

tk.Button(
    verify_frame,
    text="Verify Signature",
    command=verify_signature,
    font=("Arial", 10, "bold"),
    padx=20,
    pady=7
).pack()

# ----- Status -----

status_label = tk.Label(
    root,
    text="",
    font=("Arial", 10),
    bg="#f2f4f7"
)
status_label.pack(pady=8)

# ----- Result -----

result_label = tk.Label(
    root,
    text="",
    font=("Arial", 13, "bold"),
    bg="#f2f4f7",
    justify="center"
```

SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE

SUBJECT : Cyber & Information Security

```
)  
result_label.pack(pady=5)
```

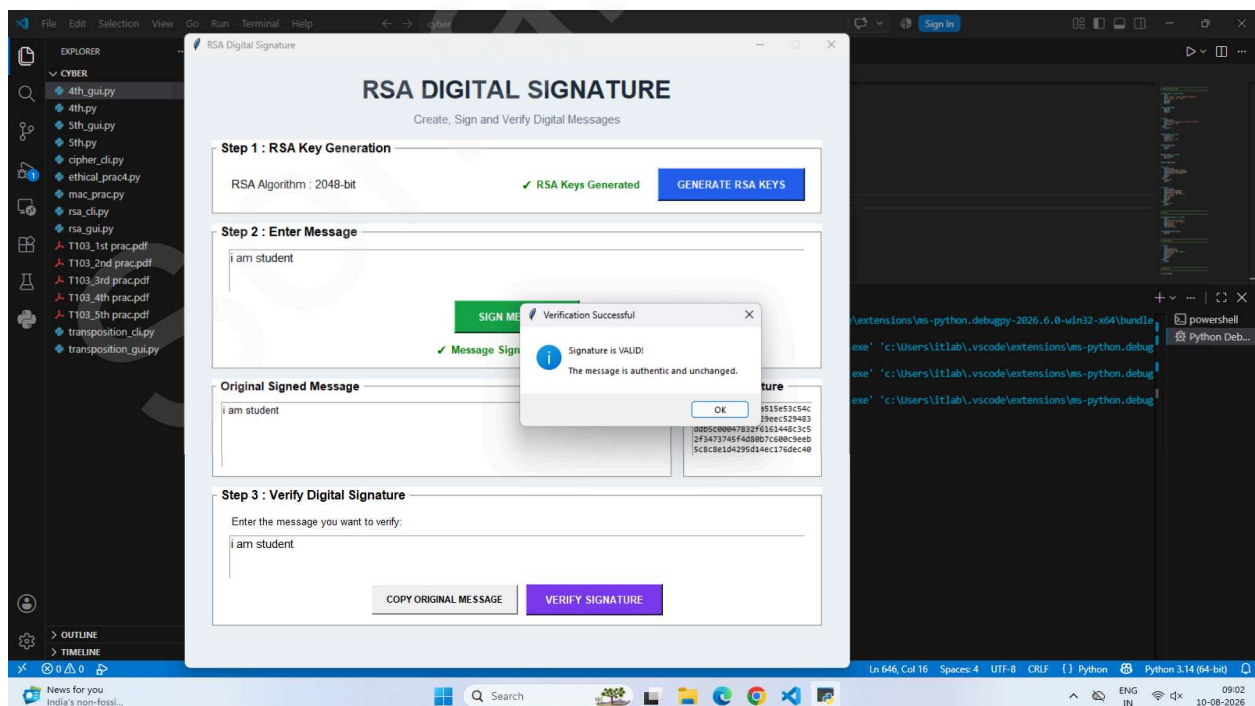
```
# ----- Clear Button -----
```

```
tk.Button(  
    root,  
    text="Clear",  
    command=clear_all,  
    font=("Arial", 10, "bold"),  
    width=15,  
    padx=5,  
    pady=7  
) .pack(pady=5)
```

```
# ----- Start Application -----
```

```
root.mainloop()
```

Output



SHETH L.U.J & SIR M.V COLLEGE OF SCIENCE

SUBJECT : Cyber & Information Security

